Relational E-matching in an SMT solver

Amar Shah
Carnegie Mellon University
Pittsburgh, USA
amarshah@andrew.cmu.edu

Marijn Heule
Carnegie Mellon University
Pittsburgh, USA
marijn@andrew.cmu.edu

Bryan Parno
Carnegie Mellon University
Pittsburgh, USA
bparno@andrew.cmu.edu

Max Willsey
University of California, Berkeley
Berkeley, USA
mwillsey@berkeley.edu

Abstract

E-matching is the problem of finding all matches of a pattern in an e-graph. Relational e-matching, a recent approach inspired by the databases community, has found succcess in equality saturation engines such as egglog. However, until now, this technique has not been implemented in a Satisfiability Modulo Theories (SMT) solver. The key difficulty of implementing relational e-matching in an SMT solver is *backtracking*, i.e., the ability to undo previous matching decisions. We provide an initial implementation of a backtrackable, relational e-matching algorithm in Sundance, an SMT solver currently in development. We describe our approach of maintaining backtrackable indices and our e-matching algorithm based on a left-deep binary join. We evaluate our implementation on a set of synthetic benchmarks and show a median speedup of 25.0× over a top-down e-matching algorithm. However, when considering a set of benchmarks from the program verifier Verus, we see a median slowdown of 1.77×. We conclude with a discussion of future work to close this gap and make relational e-matching a practical component of an SMT solver.

Keywords: e-graphs, e-matching, datalog, SMT solvers

1 Introduction

E-matching is a fundamental operation of an e-graph, used in Satisfiability Modulo Theories (SMT) solvers for quantifier instantiation and in equality saturation engines for applying rewrite rules. Traditional e-matching algorithms are top-down, starting with the outermost function and recursively matching sub-patterns.

However, recent work has proposed a new technique called relational e-matching [14], where database query execution techniques are used to match syntactic patterns in an e-graph. Relational e-matching has been in state-of-the-art equality saturation engines such as egglog [13], but until now it has not been implemented in an SMT solver. This is because existing relational e-matching implementations cannot be used out of the box in an SMT solver: they must support *backtracking*. In general, existing relational e-matching algorithms make implementation choices that are very difficult to reconcile with backtracking. For instance, egglog does e-class canonicalization using a hash cons [13]

In this work, we present our initial implementation of a backtrackable, relational e-matching algorithm in Sundance [12], an SMT solver currently in development. We describe maintaining backtrackable indices using a persistent data-structure and implement an e-matching algorithm based on a left-deep binary join.

We evaluate our implementation on a set of synthetic benchmarks designed to test e-matching and show a 25× median speedup over a single top-down e-matching algorithm. However, the story is less rosy on real workloads. On a set of benchmarks from the program verifier Verus [7], the relational e-matching algorithm provides a median slowdown of 1.77×. We believe that relational e-matching can be a practical component of an SMT solver, but it requires more work to close the gap on real workloads.

2 Background

SMT solvers [2] decide the satisfiability of a first-order logical formula. They are widely used in program verification, automated reasoning, and other areas of computer science. SMT solvers (such as Sundance) work by implementing a Boolean satisfiability (SAT) engine that searches for a satisfying assignment to the propositional skeleton of the input formula, and a theory solver that checks the validity of the theory-specific constraints, such as integer and real arithmetic, algebraic datatypes, and quantifiers.

2.1 Running example

To make SMT solving and the role of e-matching concrete, consider the SMT-LIB [1] query in Figure 1. The query declares two functions over an uninterpreted sort `U`: `g` (binary, line 2) and `h` (unary, line 3). It asserts a top-level disjunction (line 8); three ground facts (lines 9–11); and a single universally quantified clause (lines 12–13) with the nested pattern `(g (h x) x)`. The query is unsatisfiable, and a typical SMT solver discharges it as follows.

The SAT engine first decides the disjunction by case-splitting on its two disjuncts.

```
(declare-sort U 0)
(declare-fun g (U U) Bool)
(declare-fun h (U) U)
(declare-const a U)
(declare-const b U)
(declare-const c U)

(assert (or (= a b) (= a c)))
(assert (g (h a) b))
(assert (g (h a) c))
(assert (g a a))
(assert (forall (x U) (not (g (h x) x))
          :pattern ((g (h x) x))))
```

Figure 1. A simple SMT query exercising backtracking (via the disjunction) and a nested pattern pattern. Top-down e-matching scans all g-entries; relational e-matching can start from the small h-table and index-joins into g, skipping the `(g a a)` term entirely.

***Branch* `a = b`.** Asserting `a = b` merges the e-classes of `a` and `b`. Congruence closure then propagates this merge through the existing g-applications: the term `(g (h a) b)` becomes congruent to `(g (h a) a)`, so the e-graph now contains an application of g whose first argument is `(h a)` and whose second argument is in the e-class of a. The pattern `(g (h x) x)` therefore matches with `x = a`, and the quantifier instantiates to `(not (g (h a) a))`. This new lemma contradicts the term `(g (h a) a)` that congruence closure just derived in the e-graph, producing a theory conflict. Notice that we have reached a contradiction, but only in the current branch of the search. The solver records a conflict clause and backtracks to before the `a = b` decision.

***Branch* `a = c`.** This branch is similar. Merging the e-classes of `a` and `c` makes `(g (h a) c)` congruent to `(g (h a) a)`. The same pattern `(g (h x) x)` matches with `x = a`, the quantifier instantiates to `(not (g (h a) a))`, and we again obtain a theory conflict.

With both disjuncts refuted, the SAT engine has no remaining assignments to the disjunction and the solver returns `unsat`.

Takeaways. There are two key features that are important to highlight here. First, the disjunction forces the SAT engine to explore two case splits. After choosing one disjunct, instantiating quantifiers, and propagating consequences, the solver may discover a conflict and *backtrack* to try the other disjunct. Thus, all of the solver's data structures must be rolled back to the state they were in before the first disjunct.

Second, the pattern `(g (h x) x)` is a nested pattern: matching it requires finding a g-application whose first argument is an e-class that contains an h-application and whose second argument is in that same e-class. The two e-matching algorithms below take very different approaches to finding such matches.

2.2 E-matching

Top-down e-matching. Top-down e-matching [5], the classical algorithm used in most SMT solvers, walks the pattern from the top and recursively matches against the e-graph. For our pattern `(g (h x) x)`, the algorithm enumerates every e-node of the form `(g t1 t2)` and checks whether the e-class of `t1` contains an h-application `(h s)`, and whether the e-class of `s` equals the e-class of `t2`. In our running example after the merge `a = b`, the g-table contains the entries `(g (h a) b)`, `(g (h a) c)`, and `(g a a)`; the algorithm probes all three even though only the first yields a valid match. More generally, with n entries in the g-table and k entries in the h-table, top-down e-matching costs $O(n+k)$ work to locate the k relevant candidates, regardless of how small k is.

Relational e-matching. Relational e-matching [14] treats each function symbol's e-node table as a database relation and compiles the pattern into a multi-way join. For `(g (h x) x)`, the pattern is encoded as the conjunctive query:

$$h(x) = v_0 \quad \wedge \quad g(v_0,\ x) = root$$

Then the query scheduler can choose to start from either atom. Starting from the g-atom means the algorithm must scan all n entries in the g-table. Since there are fewer entries in the h-table, a better choice is to start from the h-atom: the algorithm looks up the k entries in the h-table, and for each one probes the g-table for entries whose first argument is the e-class of that h-entry. This costs $O(k \log n)$ work, which is a substantial improvement over top-down e-matching when k is small relative to n.

3 Approach

We now describe our implementation of relational e-matching in Sundance, an SMT solver currently in development [12]. The implementation spans three components: (1) two index data structures that map e-class roots to the e-nodes that reference them, (2) a pattern-flattening compiler that reduces a nested pattern to a conjunctive query over these indices, and (3) a left-deep join executor with greedy query planning, and semi-naive evaluation.

3.1 Index data structures

The core idea of relational e-matching is to precompute indices so that, given a bound variable's e-class, one can directly look up which e-nodes reference that e-class—without scanning every e-node of a given function symbol.

Sundance maintains two such indices, keyed by function symbol.

Output index. For each function symbol f, the *output index* maps an e-class root c to the set of f-e-nodes whose

output (i.e., the e-class of the term $f(\ldots)$ itself) is c. In code, `function_maps` is itself keyed by function symbol, with type

$$\text{FnSymbol} \rightarrow (\text{e-class} \rightarrow \{\text{e-node uid}\}).$$

Argument index. For each function symbol f and each argument position i, the *argument index* maps an e-class root c to the set of f-e-nodes whose i-th argument is in e-class c. In code, `function_indices` has type

$$\text{FnSymbol} \rightarrow (\text{ArgPos} \rightarrow (\text{e-class} \rightarrow \{\text{e-node uid}\}))$$

Both indices are implemented as `im::OrdMap` (a persistent balanced tree from the `im` crate [6]). When two e-classes are merged, entries from the old root are moved to the new root in both indices. Each entry carries a timestamp (the *matching round* at which it was inserted) to support the semi-naive evaluation discussed in Section 3.4.

Backtracking. At each SAT decision level, Sundance saves a snapshot of the two indices. Because `im::OrdMap` is a persistent data structure, cloning is $O(1)$ — it shares structure with the live copy. When backtracking, the solver restores the snapshot, discarding all index mutations that occurred under the popped decision level. New terms created by quantifier instantiations are then re-inserted into the restored indices, since those terms are permanent in the e-graph even though the equalities that triggered them may have been undone.

The cheap snapshots come at a cost: `im::OrdMap` is a persistent balanced tree and every insert, lookup, or removal pays the usual $O(\log n)$ tree walk. We discuss alternative implementations in Section 6.

3.2 Pattern flattening

A pattern such as `(g (h x) x)` is a nested term. Our relational matcher requires it to be flattened into a conjunction of relational atoms, such as $h(x) = v_0 \wedge g(v_0,\ x) = root$

The resulting atom list is a conjunctive query: a match must simultaneously satisfy all atoms. Each quantifier-bound variable (e.g., `x`) may appear in multiple atoms, acting as a join variable. Ground sub-terms (constants) are recorded as fixed bindings and used to constrain index lookups.

3.3 Join execution

Given the flattened atom list, the join executor produces all variable assignments that satisfy every atom simultaneously. We use a left-deep binary join: atoms are processed one at a time in a fixed order, and at each step the current set of partial bindings (the *frontier*) is extended by joining against the next atom.

Query planning. A good atom ordering avoids Cartesian products by ensuring that each new atom shares at least one variable with the frontier. Sundance uses a greedy heuristic:

1. Pick the atom whose function has the fewest e-nodes (smallest table).
2. For each subsequent position, pick the remaining atom that shares the most variables with the already-bound set. Break ties by smallest table size.

If any atom's function has zero e-nodes, the join is guaranteed empty and we short-circuit immediately.

Candidate retrieval. At each join step, for each partial binding in the frontier, we must find all e-nodes of the current atom's function that are consistent with the bound variables. This is where the index data structures pay off.

For each argument position i of the atom whose variable is already bound to some e-class c, we query function_indices$[f]$.args$[i][c]$ to retrieve candidate e-node uids. If the output variable is bound, we similarly query the output index. When multiple variables are bound, we intersect the candidate sets. If no variables are bound (as with the first atom in the join order), we fall back to a full scan of the function's e-node table.

3.4 Semi-naive evaluation

Each matching round assigns a timestamp to newly inserted index entries. Between rounds, the *matching round counter* is incremented. Semi-naive evaluation ensures that each round only discovers *new* matches by requiring at least one atom in the join to use only "delta" entries (those with timestamp $\geq$ the current matching round). For a flattened pattern with k atoms, we execute k passes: in pass i, atom i is restricted to delta entries while all other atoms use the full index. The union of results across all passes yields exactly the new matches. If no atom has any delta entries, the entire multipattern is skipped.

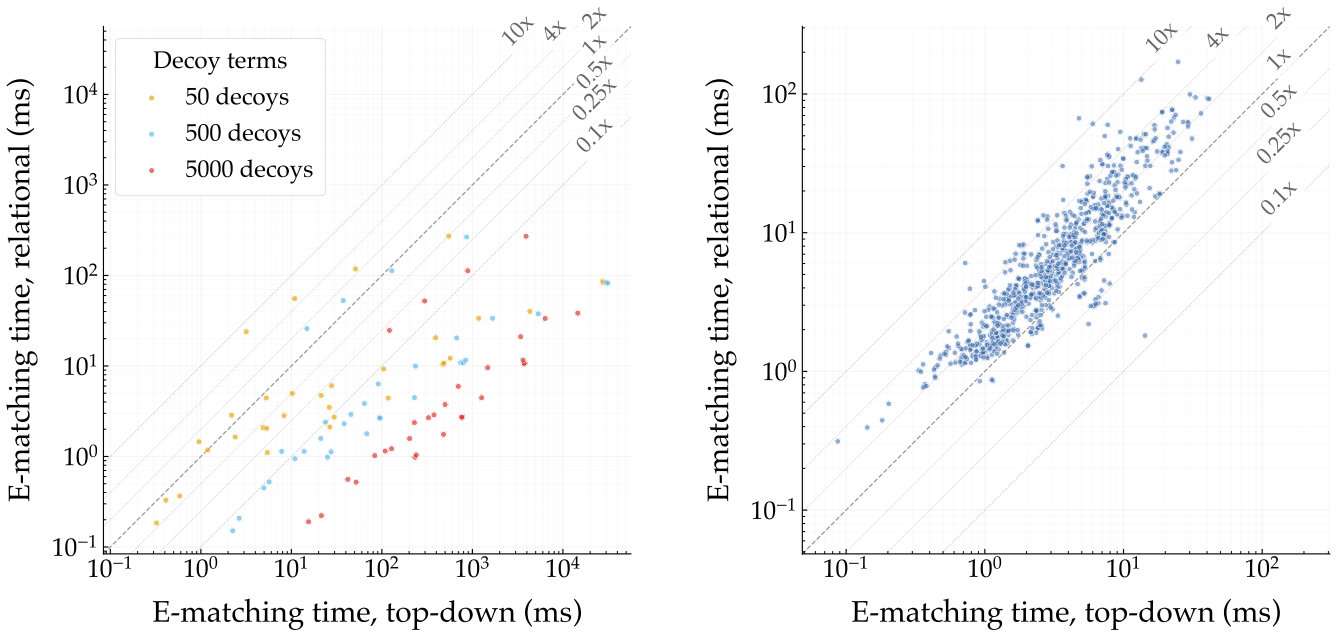
4 Evaluation

We evaluate our relational e-matching implementation against Sundance's existing top-down e-matcher on two benchmark suites: a set of synthetic queries designed to stress the patterns where relational e-matching is asymptotically advantageous, and a set of real queries from the program-verification tool Verus [7]. Figure 2 reports per-query e-matching time for both suites.

We focus on e-matching time in isolation, rather than end-to-end solve time. Our current relational e-matching implementation is a prototype with significant overhead from the `im::OrdMap` data structure and thus typically performs poorly in terms of end-to-end performance.

Specifically, on the synthetic benchmarks, out of 128 queries total, the top-down implemntation solves 123 queries (96.1 %), while the relational implementation solves 96 queries (75 %). On the Verus benchmarks, out of 3763 queries total, the top-down implementation solves 2414 queries (64.2 %), while the relational implementation solves 2247 queries (59.7 %).

These are all run with a 60 second timeout. In our analysis, we only consider queries that both implementations solve and at least one of them takes at least 1 second to solve, to



(a) Synthetic benchmarks.

(b) Verus benchmarks.

Figure 2. Per-query e-matching time, top-down (x-axis) versus relational (y-axis), in seconds. Points below the diagonal are queries on which relational e-matching is faster. Left: synthetic benchmarks designed to stress nested patterns. Right: queries from the Verus program verifier.

focus on non-trivial queries. This results in 94 queries for the synthetic suite and 908 queries for the Verus suite.

4.1 Synthetic benchmarks

The synthetic suite is a parameterized generalization of the running example from Section 2. Each query uses a nested pattern similar to `(g (h x) x)`, but could also vary in the number of quantifiers and disjunctions.

We populate the e-graph with a varying number of *decoy* entries: extra ground assertions of the form `(g d_i d_i)` that add to the g-table without contributing to any match. Top-down e-matching must scan every g-entry (decoys included) to find candidates, whereas relational e-matching starts from the comparatively small h-table and index-joins into g, touching only the relevant entries. The suite is therefore designed to reward the relational approach as the decoy count grows.

On this suite, relational e-matching is dramatically faster than top-down e-matching. Of the 94 queries solved by both matchers, relational e-matching is faster on 87, with a median speedup of 25.0× (21.3× geometric mean). Additionally, the speedup grows with the number of decoys, as expected.

4.2 Verus benchmarks

On the Verus suite, top-down e-matching is faster on average, with a median ratio of 1.77× (geometric mean 1.84×) in favor of top-down; relational e-matching is faster on only 53 of the 908 queries.

These results suggest that more work is needed to realize the asymptotic advantage of relational e-matching does not translate directly into a practical win on real. Closing this gap is the central engineering challenge identified in Section 6.

5 Related Work

E-graphs and congruence closure. The e-graph data structure was introduced by Greg Nelson in his 1980 PhD thesis [8]. It was originally designed as the core data structure of the simplifier in the Stanford Pascal Verifier, where it represented equivalence classes of ground terms under the axioms of equality and uninterpreted functions. Modern SMT solvers and equality saturation engines inherit this design.

E-matching in theorem provers. The idea of *matching modulo the equivalence stored in the e-graph* — now called e-matching — already appears in Nelson's thesis [8] . The technique was refined and used in the ESC line of program checkers, but its first detailed algorithmic exposition is the canonical Simplify paper of Detlefs, Nelson, and Saxe [5].

Indexing for SMT e-matching. de Moura and Bjørner introduced the *e-matching code tree*, a compiled abstract matching machine that shares prefix work across many patterns simultaneously, and the *inverted path index*, which makes

e-matching incremental as the e-graph grows [3]. Code trees are similar to our approach in spirit: both compile patterns to a more efficient representation than naive top-down traversal. The key difference is that code trees still drive matching from the pattern root, whereas relational e-matching is free to start from whichever relation is smallest.

Equality saturation. Equality saturation, introduced by Tate et al. [10], uses the e-graph as a compiler intermediate representation: rewrite rules are applied to saturation and an optimal program is extracted, sidestepping the phase-ordering problem in traditional optimizers. The egg library [11] made this approach widely practical by introducing deferred rebuilding of congruence invariants and e-class analyses. Relational e-matching [14] was developed in this lineage and implemented in egglog [13], where the absence of backtracking makes maintaining the relational indices straightforward.

Our contribution is to push these ideas back into the SMT setting, where backtracking is unavoidable and the index structures must be made backtrackable. Another difference is that egglog employs a worst-case optimal join algorithm [9], whereas our current implementation uses a left-deep binary join. We leave the implementation of a worst-case optimal join for future work.

6 Conclusion and Future Work

We have presented an initial implementation of relational e-matching inside an SMT solver, identifying *backtracking* as the central obstacle. Our synthetic benchmarks demonstrate that the relational approach can substantially outperform the conventional top-down matcher. Our results on Verus benchmarks suggest that it is not that simple to translate this asymptotic advantage into a practical win on real workloads. Closing the remaining gap will require a combination of better data structures, better query planning, and a tighter integration with the rest of the solver. We sketch the most promising directions below.

Better backtrackable index maps. Our current implementation maintains indices as persistent maps, which is correct but leaves substantial performance on the table. Other data structures are backtracked using trail-based undo logs, or amortized rebuilding schemes. Bringing these to bear on the relational indices would directly translate into reduced per-decision overhead.

Understanding what real benchmarks need. Synthetic benchmarks let us showcase the asymptotic advantage of relational e-matching. We do not yet have a precise characterization of which patterns and which workloads benefit, and which do not. A more careful study is needed.

Multi-query optimization. Today our implementation compiles each pattern into its own query plan and runs each plan independently. SMT solvers, however, evaluate hundreds or thousands of patterns against the same e-graph; many of these patterns share sub-patterns. The database community has long studied multi-query optimization, and Z3's code trees [3] can be viewed as exactly this kind of sharing applied to top-down e-matching. We expect that a relational analogue would yield substantial savings on real workloads.

Heuristics inside the query planner. Relational e-matching opens the door to a much richer space of optimizations than top-down matching, because the planner now has visibility into what work will be done. Many SMT-specific heuristics fit naturally into this framework. For instance, *relevancy filtering* [4] — restricting matches to terms the SAT engine currently considers active — can be expressed as an additional selection in the join plan, and a planner that knows about relevancy can prune entire join paths early. Other heuristics, such as preferring patterns that have produced useful instantiations in the past, or biasing the join order toward terms involved in recent conflicts, similarly fit as cost-model inputs to the planner.

Outlook. Indexing techniques have proven helpful in SMT performance: code trees and inverted path indices [3] are widely credited with making large-scale e-matching tractable in Z3. Relational e-matching is, in a precise sense, the most general indexing technique available — it subsumes pattern compilation as a special case and grants the planner full freedom to reorder and share work. We believe relational e-matching has a real path to becoming a practical and broadly useful component of modern SMT solvers.

References

[1] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6.* Technical Report. Department of Computer Science, The University of Iowa. https://smtlib.cs.uiowa.edu/language.shtml

[2] Clark Barrett and Cesare Tinelli. 2018. Satisfiability Modulo Theories. In *Handbook of Model Checking*. Springer, 305–343. doi:10.1007/978-3-319-10575-8_11

[3] Leonardo de Moura and Nikolaj Bjørner. 2007. Efficient E-Matching for SMT Solvers. In *Automated Deduction – CADE-21 (LNCS, Vol. 4603)*. Springer, 183–198. doi:10.1007/978-3-540-73595-3_13

[4] Leonardo de Moura and Nikolaj Bjørner. 2007. *Relevancy Propagation.* Technical Report MSR-TR-2007-140. Microsoft Research. https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-2007-140.pdf

[5] David Detlefs, Greg Nelson, and James B. Saxe. 2005. Simplify: A Theorem Prover for Program Checking. *J. ACM* 52, 3 (2005), 365–473. doi:10.1145/1066100.1066102

[6] Bodil Haines. 2023. *im-rs: Immutable Data Structures for Rust.* https://crates.io/crates/im

[7] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 85 (2023), 30 pages. doi:10.1145/3586037

[8] Charles Gregory Nelson. 1980. *Techniques for Program Verification.* Ph. D. Dissertation. Stanford University. Reissued as Xerox PARC Technical Report CSL-81-10, 1981.

[9] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2013. Skew Strikes Back: New Developments in the Theory of Join Algorithms. *SIGMOD Record* 42, 4 (2013), 5–16. doi:10.1145/2590989.2590991

[10] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality Saturation: A New Approach to Optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL).* Association for Computing Machinery, New York, NY, USA, 264–276. doi:10.1145/1480881.1480915

[11] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and Extensible Equality Saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (2021). doi:10.1145/3434304

[12] Yaspar-org. 2026. *Sundance-SMT.* https://github.com/yaspar-org/Sundance-SMT

[13] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better Together: Unifying Datalog and Equality Saturation. *Proc. ACM Program. Lang.* 7, PLDI, Article 125 (June 2023), 25 pages. doi:10.1145/3591239

[14] Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2022. Relational e-matching. *Proc. ACM Program. Lang.* 6, POPL, Article 35 (Jan. 2022), 22 pages. doi:10.1145/3498696